



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки Кафедра
інформаційних систем та технологій

Лабораторна робота №7
із дисципліни «**Технології розроблення програмного забезпечення**»
Патерни проектування.

Виконав:

студент групи ІА–34

Оборський Дмитро Вікторович

Київ 2025

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Mediator», «Facade», «Bridge», «Template method» та навчитися застосовувати їх в реалізації програмної системи.

14. Архіватор (strategy, adapter, factory method, facade, visitor, p2p)

Архіватор повинен являти собою візуальний додаток з можливістю створення і редагування архівів різного типу (.tar.gz, .zip, .rar, .ace) – додавання/ видалення файлів / папок, редагування метаданих (по можливості), перевірка checksum архівів, тестування архівів на наявність пошкоджень, розбиття архівів на частини.

Теоретичні відомості

1.1.1. Шаблон «Mediator»

Призначення патерну: Шаблон «Mediator» (посередник) використовується для визначення взаємодії об'єктів за допомогою іншого об'єкта (замість зберігання посилань один на одного) [6]. Даний шаблон схожий на шаблон «команда», проте в даному випадку замість зберігання даних про конкретну дію, зберігаються дані про взаємодії між компонентами.

Даний шаблон зручно застосовувати у випадках, коли безліч об'єктів взаємодіє між собою деяким структурованим чином, однак складним для розуміння. У такому випадку вся логіка взаємодії виноситься в окремий об'єкт. Кожен із взаємодіючих об'єктів зберігає посилання на об'єкт «медіатор».

«Медіатор» нагадує диригента при управлінні оркестром. Диригент стежить за тим, щоб кожен інструмент грав в правильний час і в злагоді з іншими інструментами. Функції «медіатора» повністю це повторюють.

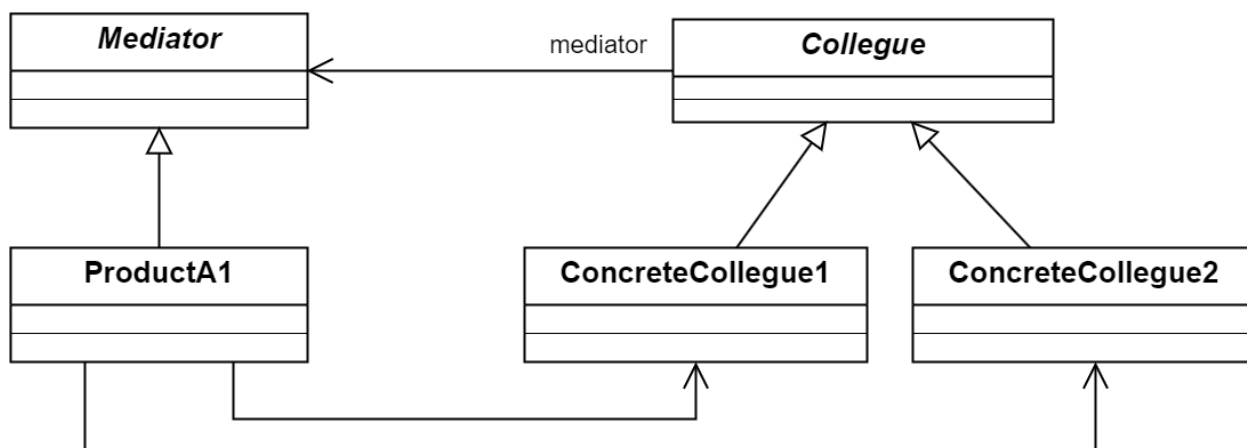


Рисунок 7.1. Структура патерна «Медіатор»

Проблема: Ви розробляєте систему зі складною візуальною частиною. Головна форма з якою працює користувач складається з великою кількості візуальних компонентів, які в свою чергу складаються з елементарних візуальних компонентів. Для такої складної форми, як правило, є багато логіки, коли дії в одному компоненті повинні впливати на інші компоненти.

Наприклад, коли ви вибрали чекбокс, то після цього відключається можливість редагування даних в деяких частинах форми, деякі блоки мають приховатися, а інші навпаки, показатися. В такій ситуації, коли на формі може бути сотня, а то і більше візуальних контролів, то зв'язки між ними можуть вимірюватися тисячами. Додавати зміни в функціонал на таких формах дуже складно, а знайти та виправити помилку, взагалі, може бути дуже затратно по витраченому часу.

Рішення: В таких ситуаціях дуже добре підходить патерн «Mediator» (Посередник). При реалізації такого патерна, ми додаємо новий клас посередник, і всі взаємодії між компонентами повинні відбуватися вже через нього. За рахунок цього компоненти вже не знають один про одного, а лише

знають про посередника і для відпрацювання логіки звертаються вже до нього. А вже посередник взаємодіє з іншими компонентами для відпрацювання цієї логіки.

Основною перевагою, яку ми отримуємо при такому підході є сильно зменшена зв'язність між компонентами. А за рахунок цього стає набагато простіше розібратися з взаємодією між об'єктами коли потрібно додати на форму нові об'єкти або додати якийсь функціонал. Якщо компоненти будуть взаємодіяти не напряму з посередником, а через загальний інтерфейс посередника, то тоді можна буде підміняти посередника, наприклад, на тестового, щоб емулювати тестові випадки для перевірки.

Переваги та недоліки:

- + Організація взаємодії між об'єктами лише через посередника спрощує розуміння та супроводження такого коду.
- + Додавання нових посередників без зміни існуючих компонентів дозволяє розширювати систему без зміни існуючого коду.
- + Зменшення залежностей між об'єктами підвищує гнучкість системи.
- Посередник, з часом, може перетворитися на дуже складний об'єкт, який робить все («God Object»).

1.1.2. Шаблон «Facade»

Призначення патерну: Шаблон «Facade» (фасад) передбачає створення єдиного уніфікованого способу доступу до підсистеми без розкриття внутрішніх деталей підсистеми [6]. Оскільки підсистема може складатися з безлічі класів, а кількість її функцій – не більше десяти, то щоб уникнути створення «спагеті-коду» (коли все тісно пов'язано між собою) виділяють один загальний інтерфейс доступу, здатний правильним чином звертатися до внутрішніх деталей.

Це також відволікає користувачів від змін в підсистемі (внутрішня реалізація може змінюватися, а наданої послуги немає), що також скоротить кількість змін в використовуваних фасад класах (без фасаду довелося б змінювати вихідні коди в безлічі точок).

Звичайно, твердої умови повного закриття внутрішніх класів підсистеми не стоїть – при необхідності можна звертатися до окремих класів безпосередньо, минаючи об'єкт фасад.

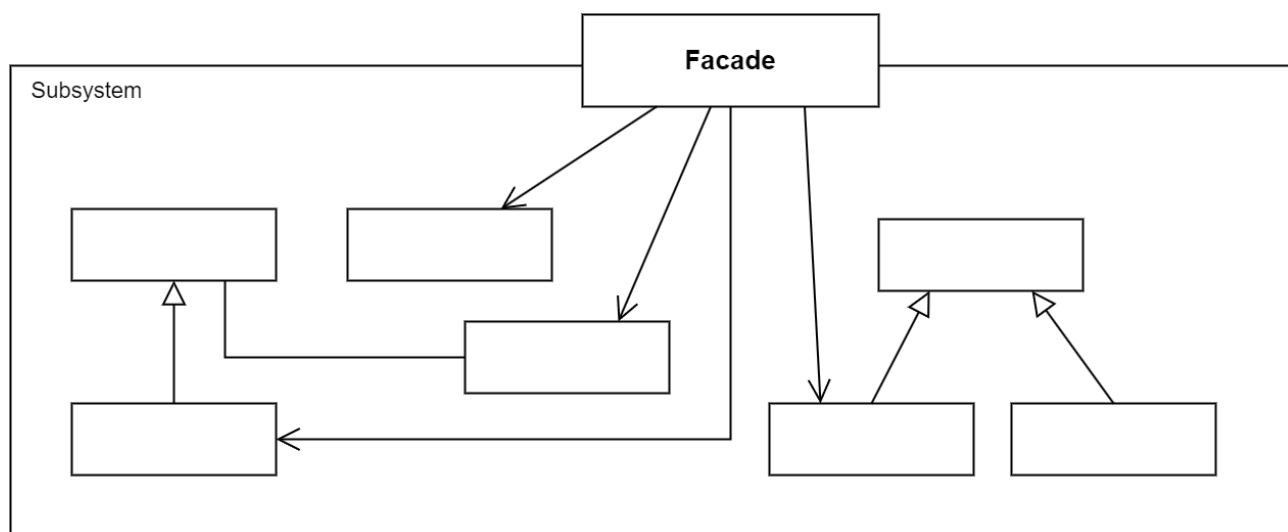


Рисунок 7.2. Структура патерну «Фасад»

Проблема: Ви розробляєте компонент, який дозволяє відправляти запити на різні типи endpoint, а також працює з протоколами HTTP та TCP/IP.

Прототип компонента вже працює, але структура класів вийшла досить складна, а при налаштуванні на різні протоколи мають використовуватися різні класи.

Інструкція для використання також виходить досить складна та заплутана. Слід додати, так як інші системи будуть знати про внутрішню будову вашого компонента, а тому при спробі змінити внутрішню структуру в наступних версіях, вам прийдеться повідомляти про це всіх користувачів вашого компонента і для них перехід на наступну версію вашого компонента буде достатньо складним.

Рішення: Тут краще використати патерн фасад. А саме, в даному випадку, створити один клас, наприклад, `InternetClient`, та набір методів у нього, які будуть використовуватися для налаштування цього підключення, та його подальшого використання. Цей клас єдиний буде позначено як `public`, і тільки його будуть

бачити ваші клієнти. Таким чином, з точки зору зовнішнього коду вони будуть працювати з набагато простішим інтерфейсом, а значить і інструкція використання буде значно простіша. Крім того, так як внутрішня структура повністю закрита, то в наступних версіях ви можете її змінювати, як вам буде зручно, а з точки зору користувачів компонента, перехід на нову версію буде просто переключенням на нову версію бібліотеки.

Переваги та недоліки:

- + Інкапсуляція внутрішньої структури від клієнтського коду.
- + Спрощується інтерфейс для роботи з модулем закритим фасадом.
- Зниження гнучкості в налаштуванні та використанні програмного коду закритого фасадом.

1.1.3. Шаблон «Bridge»

Призначення патерну: Шаблон «Bridge» (міст) використовується для поділу інтерфейсу і його реалізації. Це необхідно у випадках, коли може існувати кілька різних абстракцій, над якими можна проводити дії різними способами.

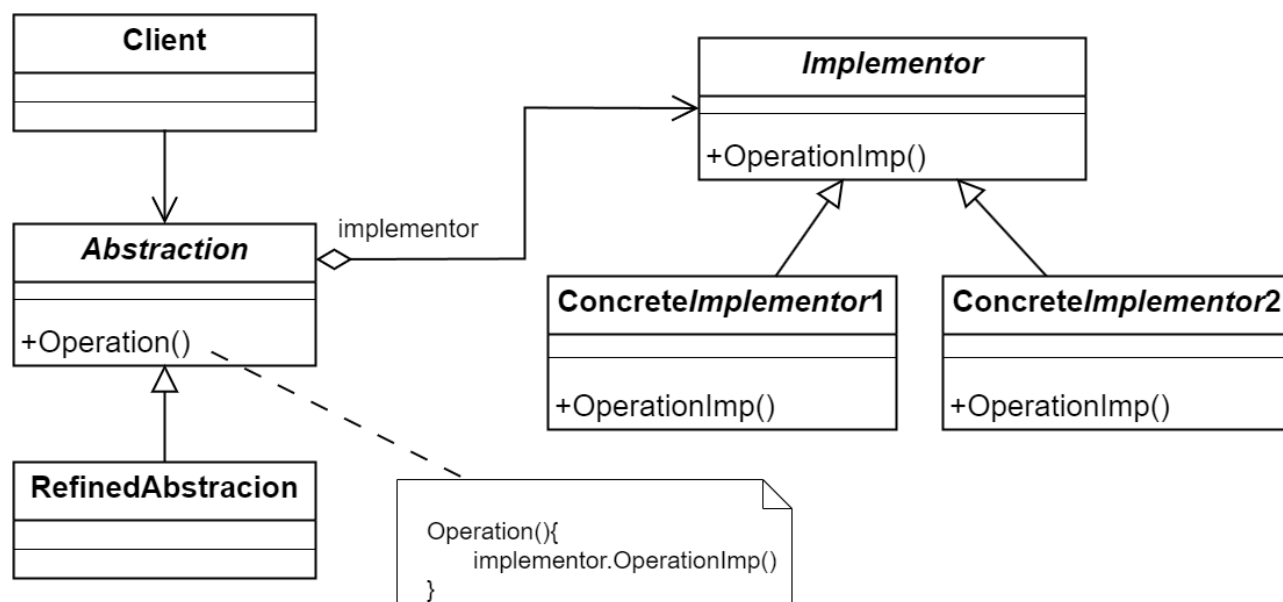


Рисунок 7.3. Структура патерну «Міст»

Проблема: Ви реалізовуєте графічний векторний редактор, який дозволяє рисувати кола, прямокутники, прямі та довільні лінії.

Ви маєте реалізувати функціонал відображення отриманого рисунку на екрані та друкувати на принтер.

Спростимо ситуацію: ваш редактор дозволяє рисувати лише лінії та кола.

При такому підході, нам потрібно будувати ієрархію фігур з різною реалізацією дочірніх класів: LinePrint, LineDraw, CirclePrint, CircleDraw.

якщо додати прямі, то добавиться ще два підкласи, і т.д. А як бути, коли нам потрібно буде ще і реалізувати збереження в bitmap форматі? додаємо ще LineBinary, CircleBinary? При такому підході ми отримуємо дуже складну ієрархію класів.

Рішення: В даному випадку ми можемо використати патерн «Міст» (Bridge): робимо дві ієрархії – фігур (Shape) та рисунка (DrawApi).

При такому підході DrawApi – це інтерфейс імплементації відображення (графічного драйвера), а Shape – інтерфейс абстракції фігур, яка має агрегацію з об'єктом DrawApi. При такому підході фігури будуть делегувати рисунка об'єкту DrawApi.

Лінія, коло, та інші будуть дочірніми класами до Shape, а WindowDrawApi та PrinterDrawApi – дочірні класи до DrawApi, які представляють графічні драйвери для відображення на екрані та принтері відповідно. Якщо нам потрібно буде додати ще і збереження в bitmap форматі, то ми добавимо ще один підклас реалізації графічного драйвера BitmapDrawApi. Таким чином ми маємо дві різні ієрархії об'єктів і вони в нас не перетинаються і не збільшуються в геометричній прогресії при додаванні нових драйверів або фігур.

Також слід відмітити, що DrawApi нічого не знає про фігури (абстракцію), а дочірні класи абстракції не залежать від реалізації графічного драйвера.

Переваги та недоліки:

+ Дозволяє змінювати ієрархії абстракції та реалізації незалежно одна від одної.

+ Розділивши абстракцію від реалізації отримуємо більшу гнучкість та простіший супровід такого коду.

- Підвищена гнучкість при використанні патерну отримується за рахунок більшої складності, введення додаткових проміжних рівнів.

1.1.4. Шаблон «Template Method»

Призначення патерну: Шаблон «Template Method» (шаблонний метод) дозволяє реалізувати покроково алгоритм в абстрактному класі, але залишити специфіку реалізації підкласам [6]. Можна привести в приклад формування веб-сторінки: необхідно додати заголовки, вміст сторінки, файли, що додаються, і нижню частину сторінки. Код для додавання вмісту сторінки може бути абстрактним і реалізовуватися в різних класах – `AspNetCompiler`, `HtmlCompiler`, `PhpCompiler` і т.п. Додавання всіх інших елементів виконується за допомогою вихідного абстрактного класу з алгоритмом.

Даний шаблон дещо нагадує шаблон «Фабричний метод», однак область його використання абсолютно інша – для покрокового визначення конкретного алгоритму; більш того, даний шаблон не обов'язково створює нові об'єкти – лише визначає послідовність дій.

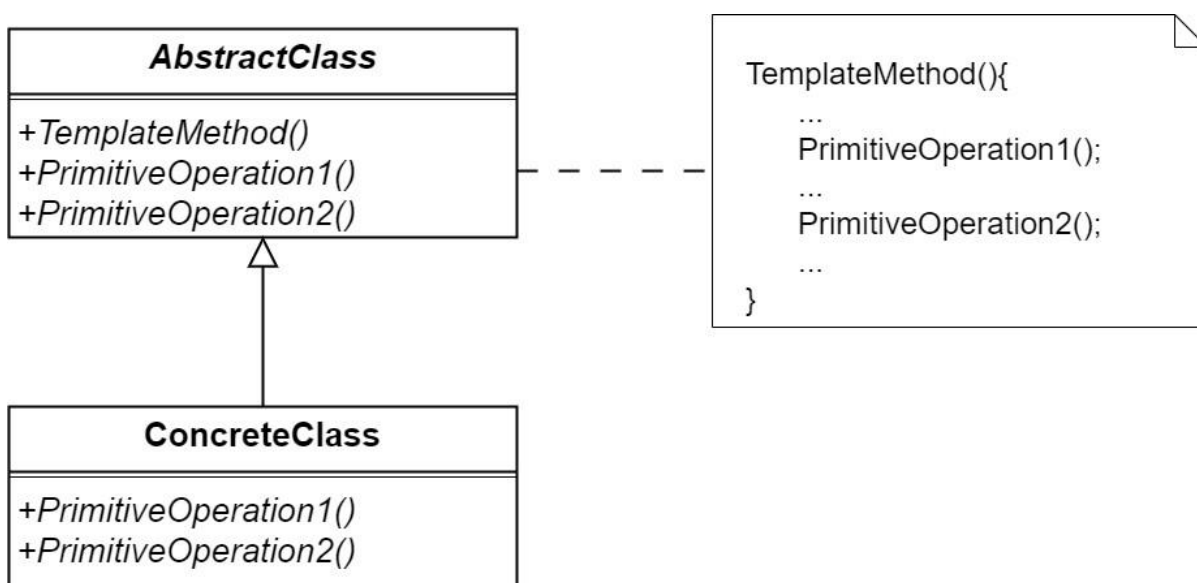


Рисунок 7.4. Структура патерну «Шаблонний метод»

Проблема: Ви працюєте в команді, що займається розробкою застосунку для редагування відео-файлів. Застосунок вже працює з форматом відео MPEG-4, а саме дозволяє читати такі файли, виконувати попередню обробку даних для відображення в відео-редакторі.

Ви отримуєте нову задачу на реалізацію можливості роботи з більш старим форматом MPEG-2. Ви бачите два варіанта: зробити копію існуючого класу, що працює з MPEG-4, або вносити зміни в уже існуючий клас. Щоб прийняти рішення ви більш детально розбираєтеся з існуючим алгоритмом і бачите, що близько 70 відсотків коду має бути таким самим. Тому ви вирішуєте змінити вже існуючий клас для роботи з MPEG-4 додаваючи в місцях де це потрібно умови з перевіркою, що якщо формат MPEG-2 то відпрацьовувати новий код, який ви добавили. Через деякий час, на запити від користувачів, вам на реалізацію приходить задача добавити підтримку ще більш старого формату MPEG-1. Ви вносите зміни так само в існуючий клас, тільки умови стали більш складними, тому що розгалуження логіки йде на три гілки.

Ще через деякий час приходить аналогічна задача на додавання читання даних з файлів формату H.262. Ви починаєте працювати над задачею і бачите, що код, який до цього був ще більш-менш зрозумілим стає зовсім важким для читання та внесення змін.

Рішення: Патерн «Шаблонний метод» (Template Method) пропонує загальний алгоритм винести в базовий клас, а частини алгоритма, які для різних задач виконуються по різному, виділити в окремі методи. Ці методи будуть викликатися в алгоритмі, що реалізований в базовому класі. В дочірніх класах ці виділені методи будуть перевизначатися. Таким чином загальна логіка залишається в базовому класі, а специфічна частина реалізується в дочірніх класах.

Якщо подивитися на задачу з відео-редактором, то застосування «Шаблонного методу» наведе лад в коді і спростить його зміни.

Як це зробити: По перше, в алгоритмі всі блоки коду де є вибір гілки на основі типу формату виділяються в окремі методи. У випадку з відео-редактором, це скоріш за все будуть блоки коду пов'язані з читанням даних та розпакування їх в кадри, а також читання звукових доріжок. Далі створюється загальний базовий клас в який переноситься загальний алгоритм, а також об'являються віртуальні методи (фактично беремо сигнатуру тих методів, що виділили на попередньому кроці). Далі створюємо дочірні класи під кожен формат файлу і перевизначаємо віртуальні методи. Фактично при цьому в кожному такому методі в дочірньому класі із реалізації цих методів, що була виділена на першому кроці, залишається код гілки який відповідав вибраному формату.

Після всіх цих змін ми маємо реалізацію патерна «Шаблонний метод»: в базовому класі реалізовано базовий алгоритм (по суті більша частина алгоритму) і в дочірніх класах перевизначені методи зі специфічною логікою.

Після таких змін, додати підтримку нового формату стає легше, тому що достатньо буде додати лише новий дочірній клас і перевизначити в ньому необхідні методи.

Слід зауважити, що якщо у вас алгоритми співпадають більше ніж на 50 відсотків, то застосування шаблонного методу буде доцільним, але якщо у вас алгоритми співпадають лише відсотків на 10 або 20, то скоріш за все, краще буде використати патерн «Стратегія».

Переваги та недоліки:

- + Полегшує повторне використання коду.
- Ви жорстко обмежені скелетом існуючого алгоритму.
- Ви можете порушити принцип підстановки Барбери Лісков, змінюючи базову поведінку одного з кроків алгоритму через підклас.
- З ростом складності загального алгоритму шаблонний метод стає занадто складно підтримувати, особливо, коли є багато віртуальних методів для перевизначення в підкласах.

Хід роботи

1. Створюю фасад ArchiveService для ArchiverFactory з попередньої роботи

```

1 package com.beowulf.core.facade;
2
3 import com.beowulf.core.factory.ArchiverFactory;
4 import com.beowulf.core.interfaces.Archiver;
5
6 import java.io.IOException;
7 import java.nio.file.Path;
8
9 You, 4 days ago | 1 author (You)
10 public class ArchiveService {
11     private final ArchiverFactory factory = new ArchiverFactory();
12
13     /**
14      * Compress a source directory or file into a single archive file.
15      *
16      * @param sourceDir path to file or directory to compress
17      * @param targetArchive path to resulting archive file
18      */
19     public void compress(Path sourceDir, Path targetArchive) throws IOException {
20         Archiver archiver = factory.getArchiver(targetArchive);
21         archiver.compress(sourceDir, targetArchive);
22     }
23
24     /**
25      * Decompress an archive file into a target directory.
26      *
27      * @param archive path to existing archive
28      * @param targetDir destination directory
29      */
30
31     public void decompress(Path archive, Path targetDir) throws IOException {
32         Archiver archiver = factory.getArchiver(archive);
33         archiver.decompress(archive, targetDir);
34     }
35 }

```

Рис. 1 ArchiveService

2. Розроблюю діаграму класів

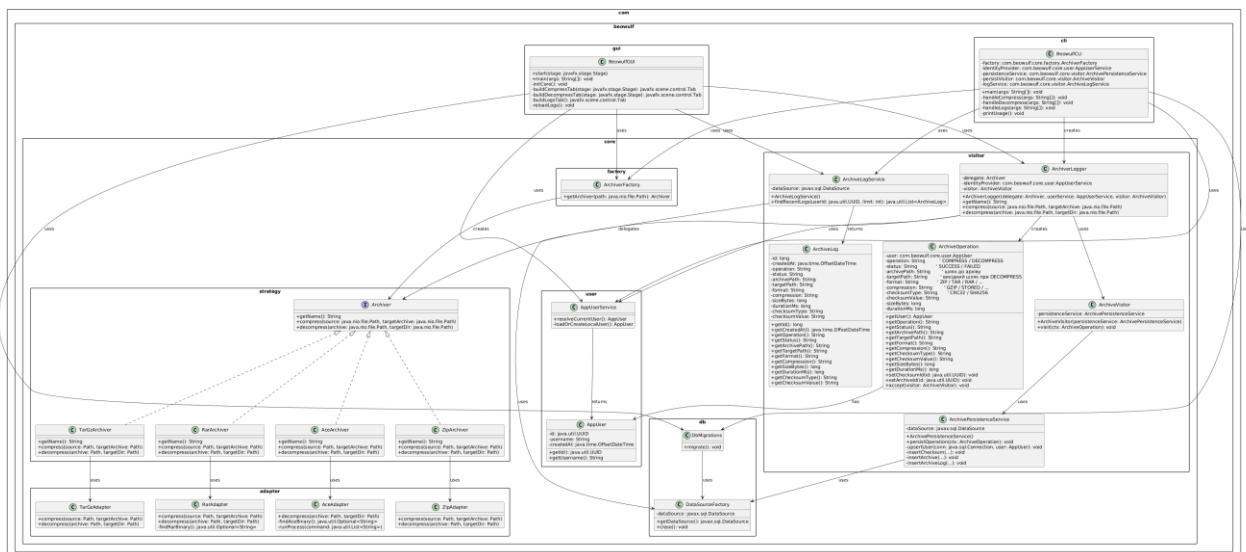


Рис. 2 Діаграма класів

Висновок

У ході цієї лабораторної роботи я поглибив свої знання про шаблони «Посередник», «Фасад», «Міст» та «Шаблонний метод» і розглянув їх роль у побудові гнучкої архітектури програмних систем. Я проаналізував їх структуру, принципи взаємодії між класами та особливості використання в реальних застосуваннях. Зокрема, було з'ясовано, як «Посередник» дозволяє зменшити зв'язність між компонентами, «Фасад» спрощує взаємодію з підсистемами, «Міст» забезпечує незалежний розвиток абстракцій та реалізацій, а «Шаблонний метод» задає сталу структуру алгоритму з можливістю перевизначення окремих кроків. Отримані знання продемонстрували, що правильне застосування цих патернів робить програмне забезпечення більш стійким до змін, зрозумілішим у підтримці та придатним до масштабування.

Контрольні запитання

1. Яке призначення шаблону «Посередник»?

(Зменшити зв'язність між об'єктами, винісши їхню взаємодію в окремий об'єкт-посередник, щоб компоненти не залежали напряму один від одного)

2. Нарисуйте структуру шаблону «Посередник».

(Складається з інтерфейсу Mediator, конкретного посередника та набору компонентів (Colleagues), які взаємодіють між собою тільки через Mediator)

3. Які класи входять в шаблон «Посередник», та яка між ними взаємодія?

- ✧ Mediator — описує методи взаємодії між компонентами.
- ✧ ConcreteMediator — реалізує логіку координації.
- ✧ Colleague — компонент, який знає лише про посередника.
- ✧ ConcreteColleague — надсилає події посереднику.

(Взаємодія: компоненти не спілкуються напряму — тільки через mediator)

4. Яке призначення шаблону «Фасад»?

(Надати спрощений, єдиний інтерфейс до складної підсистеми, приховуючи від клієнта внутрішні деталі її роботи)

5. Нарисуйте структуру шаблону «Фасад».

(Є клас Facade, який містить та викликає декілька класів підсистеми, надаючи один узагальнений метод для роботи з ними)

6. Які класи входять в шаблон «Фасад», та яка між ними взаємодія?

- ✧ Facade — точка входу, викликає методи підсистеми.
- ✧ SubsystemClasses — реальні компоненти системи.

(Взаємодія: клієнт працює тільки з Facade, а той — з підсистемою)

7. Яке призначення шаблону «Міст»?

(Розділити абстракцію та її реалізацію так, щоб вони могли розвиватися незалежно одна від одної)

8. Нарисуйте структуру шаблону «Міст».

(Складається з Abstraction, який містить посилання на Implementor, та двох ієрархій — абстракцій і реалізаторів)

9. Які класи входять в шаблон «Міст», та яка між ними взаємодія?

- ✧ Abstraction — описує базову логіку та містить посилання на Implementor.
- ✧ RefinedAbstraction — розширена абстракція.
- ✧ Implementor — інтерфейс реалізації.
- ✧ ConcreteImplementor — конкретні реалізації.

(Взаємодія: Abstraction делегує роботу Implementor, зберігаючи незалежність обох сторін)

10. Яке призначення шаблону «Шаблонний метод»?

(Відповідь) Визначити структуру алгоритму в базовому класі, делегувавши окремі кроки підкласам через методи, які вони можуть перевизначати.

11. Нарисуйте структуру шаблону «Шаблонний метод».

(Відповідь) Є базовий клас із методом-шаблоном (templateMethod), який викликає абстрактні та конкретні підметоди; підкласи реалізують абстрактні кроки.

12. Які класи входять в шаблон «Шаблонний метод», та яка між ними взаємодія?

✧ AbstractClass — містить загальний алгоритм (templateMethod).

✧ ConcreteClass — реалізує окремі кроки алгоритму.

(Взаємодія: клієнт викликає templateMethod, а конкретні кроки виконуються підкласами)

13. Чим відрізняється шаблон «Шаблонний метод» від «Фабричного методу»?

✧ Шаблонний метод задає алгоритм і делегує певні його частини підкласам.

✧ Фабричний метод делегує створення об'єктів підкласам.

(Тобто Template Method — про логіку; Factory Method — про створення)

14. Яку функціональність додає шаблон «Міст»?

(Дозволяє незалежно змінювати абстракцію та її реалізацію, комбінувати різні варіанти обох сторін та уникати жорсткого зв'язку між ними)